SOFTWARE TESTING & VALIDATION · FINAL PROJECT

EasyService: Test Plan — Part A (v1.0)

Institution: Addis Ababa University — School of Information Technology and Engineering

Course: Software Testing and Validation

Instructor: Abel Tadesse

Deadline: Sunday, 13 September 2026, end of day

Document: Test Plan (Part A) — v1.0

Repository: https://github.com/Abyman2/Easyservice

Group Members

#	Full Name	Student Number	Email Address	GitHub Username
1	**Abel Seleshe**	ATE/6743/14	abelseleshi24@gmail.com	[Abyman2](https://github.com/Abyman2)
2	**Baheran Tesfaye**	ATE/5750/14	Bahrantesfaye1@gmail.com	[Bahrann](https://github.com/Bahrann)
3	**Nebiyu Yohannes**	ATE/3973/14	natijhonny@gmail.com	[NebiyuYohannes](https://github.com/NebiyuYohannes)
4	**Wondesen Teshale**	ATE/4671/14	wendesentesha16@gmail.com	[WondesenTeshale](https://github.com/WondesenTeshale)

Table of Contents

---

1. Introduction & Purpose

This Test Plan governs the testing effort for **EasyService**, the group's final project for Software Testing and Validation. EasyService is an Ethiopia-first marketplace connecting customers with service providers across hotel rooms, car rentals, event tickets, and store products.

The objective of this document and testing effort is to systematically apply formal software testing principles:

- Black-box test design (Equivalence Partitioning, Boundary Value Analysis, Decision Tables, State Transition Testing).

- Multi-tiered automated testing (Unit with JUnit 5 + Mockito, Integration with Spring Boot Test + MockMvc, System/E2E with Selenium WebDriver).

- Code coverage analysis targeting ≥**80% branch coverage** via JaCoCo.

- Continuous Integration via GitHub Actions and Jenkins.

- Defect tracking and quality metrics calculation for release recommendation.

---

2. Scope

EasyService relies on a single shared marketplace engine:

$$\text{Listing} \rightarrow \text{Transaction} \rightarrow \text{Payment} \rightarrow \text{Transaction State}$$

In scope: Customer registration, Fayda/Passport identity verification, authentication, listing management, booking/purchasing, promotion application, simulated payment processing, cancellation, and transaction state machine transitions.

---

3. Test Items — In Scope & Out of Scope

3.1 In Scope

- Customer Registration (Ethiopian & Foreign types).

- Authentication (Email or Phone + Password).

- Identity Verification (Simulated Fayda / Passport).

- Listing Browse, Search, and Category Filtering.

- Booking, Renting, and Product Purchase.

- Promotion Application (Discount % and Minimum Threshold).

- Payment Processing (FakePaymentService).

- Transaction Cancellation and Inventory Restoration.

- Provider Dashboard and Listing Management.

3.2 Out of Scope

Category	Excluded	Reason
:---	:---	:---
Payments	Real Telebirr, CBE, or Credit Cards	Academic project safety; fake doubles isolate test state
Identity	Real Fayda API or Passport Systems	Avoid storing or processing real PII
Data	External Relational Database	In-memory repositories provide deterministic state
Features	OCR Receipt Scanning, AI Recommendations	Time-boxed; priority given to testing depth

---

4. Approach — Levels & Techniques

4.1 Test Levels (The Pyramid)

1. **Unit Tier (JUnit 5 + Mockito)**: High volume. Tests business rules, pricing, availability, identity rules, state transitions in isolation.

2. **Integration Tier (Spring Boot Test + MockMvc)**: Medium volume. Validates Controller $\rightarrow$ Service $\rightarrow$ Repository interaction.

3. **System Tier (Selenium WebDriver + POM)**: Low volume (8–12 E2E scenarios). Validates full user browser journeys.

4.2 Formal Techniques

- **Equivalence Partitioning (EP)**: Quantity, balance, identity validity.

- **Boundary Value Analysis (BVA)**: Listing capacity boundaries, minimum promo thresholds, exact balance payment.

- **Decision Tables**: Booking eligibility ($\text{Identity Verified} \times \text{Listing Published} \times \text{Availability} \times \text{Payment}$).

- **State Transition Testing**: Valid state transitions ($\text{PENDING} \rightarrow \text{CONFIRMED} \rightarrow \text{COMPLETED}$) and explicit negative tests for forbidden transitions ($\text{COMPLETED} \rightarrow \text{CANCELLED}$).

---

5. Features to be Tested / Not Tested

- Registration & Identity Validation: **Tested (Unit, Integration)**

- Login (Email/Phone): **Tested (Unit, Integration, Selenium)**

- Listing Publish/Unpublish: **Tested (Unit, Integration)**

- Booking & Availability Rules: **Tested (Unit, Integration, Selenium)**

- Simulated Payment: **Tested (Unit, Integration)**

- Promotion Application: **Tested (Unit, Integration)**

- Cancellation & Inventory Restoration: **Tested (Unit, Integration)**

- Transaction State Machine: **Tested (Unit)**

- Bill Split & Random Payer Wheel: **Tested (Unit)**

---

6. Risk-Based Prioritisation

- **High Priority**: Payment & balance logic, identity verification, transaction state machine, inventory/capacity rules.

- **Medium Priority**: Provider ownership boundaries, promotion date/amount thresholds, category filtering.

- **Low Priority**: Easy Tools (Bill Split, Wheel), UI cosmetics.

---

7. Entry & Exit Criteria

Entry Criteria

- Business rules BR-01 to BR-20 specified and numbered.

- Code compiles clean without errors.

- Test doubles (`FakePaymentService`, `FakeIdentityVerificationService`, `FakeNotificationService`) available.

Exit Criteria

- 100% of in-scope features have passing unit/integration tests.

- ≥**80% branch coverage** verified by JaCoCo on core logic packages.

- All Selenium E2E scenarios pass.

- Green GitHub Actions and Jenkins pipelines.

- Zero open Critical defects.

---

8. Test Environment & Tools

- **Language & Framework**: Java 21, Spring Boot 3.3.2.

- **Testing Tools**: JUnit 5, Mockito, Spring Boot Test, MockMvc, Selenium WebDriver.

- **Coverage**: JaCoCo Maven Plugin.

- **CI/CD**: GitHub Actions, Jenkins (via Docker).

---

9. Schedule & Roles

- **Week 1**: Architecture, core domain, Test Plan (Part A), initial CI setup.

- **Week 2**: Unit/Integration test suites, EP/BVA derivation (Part B), 80% branch coverage target.

- **Week 3**: System E2E Selenium tests, defect log & metrics (Part F/G), Test Summary Report (Part H/I), final submission.

Roles:

- **Cha**: Backend & Automation Lead (Spring Boot, JUnit 5, JaCoCo, CI/CD).
- **Abel**: Frontend & System Test Lead (Selenium, POM, E2E Scenarios).
- **Baheran**: Test Design & Quality Lead (Test Plan, EP/BVA, Defect Log, Metrics, Summary Report).

---

10. Concept Coverage Cross-Reference

- Error, Fault, Failure: Covered in Test Summary Report & Defect Log.
- Test Design Techniques: Test Design Document (Part B).
- Coverage: JaCoCo report in CI (Part D).
- Test Pyramid & Doubles: Unit Suite (Part C).
- Continuous Integration: GitHub Actions & Jenkins (Part E).

---

11. Approval & Sign-Off

- **Cha**: Backend & Automation Lead — *Approved* (2026-09-02)
- **Abel**: Frontend & System Test Lead — *Approved* (2026-09-02)
- **Baheran**: Test Design & Quality Lead — *Approved* (2026-09-02)